Unit - II

Control Flow Statements: if, if-else, if-elif-else, while loop, range() Function, for loop, continue and break statements.

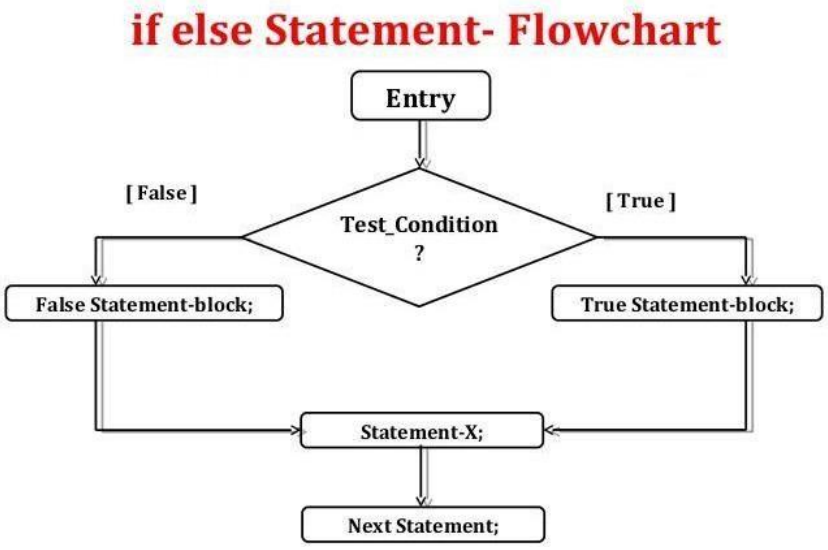
Strings: Creating and Storing Strings, Basic String Operations, String Slicing and Joining, String Methods, Formatting Strings

Control Flow Statements

1. **Decision Making and branching (Conditional Statement)**
2. **Looping or Iteration**
3. **Jumping statements**

DECISION MAKING & BRANCHING

Decision making is about deciding the order of execution of statements based on certain conditions. Decision structures evaluate multiple expressions which produce TRUE or FALSE as outcome.



There are three types of conditions in python:

1. if statement
2. if-else statement
3. elif statement

1. **if statement:** It is a simple if statement. When condition is true, then code which is associated with if statement will execute.

Example:

```
a = 40
b = 20
if  a>b:
        print(“a is greater than b”)
```

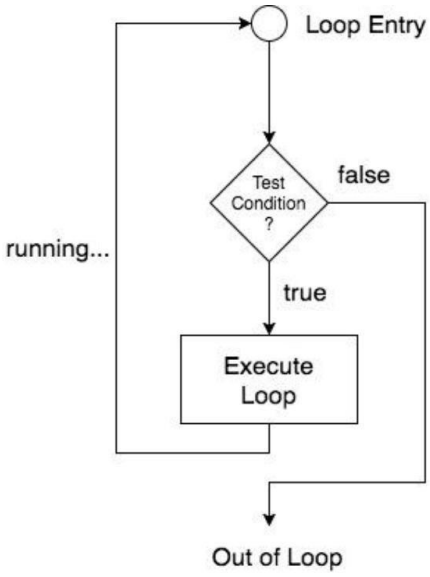
2. if-else statement: When the condition is true, then code associated with if statement will execute, otherwise code associated with else statement will execute.

Example:

```
a = 10
b = 20
if a>b:
        print(“a is greater”)
elif a==b:
        print("both numbers are equal")
else:
        print("b is greater")
```

LOOPS in PYTHON

Loop: Execute a set of statements repeatedly until a particular condition is satisfied



There are two types of loops in python:

1. while loop
2. for loop

Python offers 2 choices for running the loops. The basic functionality of all the techniques is the same, although the syntax and the amount of time required for checking the condition differ.

We can run a single statement or set of statements repeatedly using a loop command.

The following sorts of loops are available in the Python programming language.

Sr.No.	Name of the loop	Loop Type & Description
1	**While loop**	Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
2	**For loop**	This type of loop executes a code block multiple times and abbreviates the code that manages the loop variable.

Loop Control Statements

Statements used to control loops and change the course of iteration are called control statements. All the objects produced within the local scope of the loop are deleted when execution is completed.

Python provides the following control statements. We will discuss them later in detail.

Let us quickly go over the definitions of these loop control statements.

Sr.No.	Name of the control statement	Description
1	**Break statement**	This command terminates the loop's execution and transfers the program's control to the statement next to the loop.
2	**Continue statement**	This command skips the current iteration of the loop. The statements following the continue statement are not executed once the Python interpreter reaches the continue statement.
3	**Pass statement**	The pass statement is used when a statement is syntactically necessary, but no code is to be executed.

The while Loop

With the while loop we can execute a set of statements as long as a condition is true.

Example

Print i as long as i is less than 6:

```
i = 1
while i < 6:
  print(i)
  i += 1
```

Note: remember to increment i, or else the loop will continue forever.

The while loop requires relevant variables to be ready, in this example we need to define an indexing variable, i, which we set to 1.

The break Statement

With the break statement we can stop the loop even if the while condition is true:

Example

Exit the loop when i is 3:

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

The continue Statement

With the continue statement we can stop the current iteration, and continue with the next:

Example

Continue to the next iteration if i is 3:

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```

The else Statement

With the else statement we can run a block of code once when the condition no longer is true:

Example

Print a message once the condition is false:

```
i = 1
while i < 6:
```

```
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

Python For Loops

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Syntax of the for Loop

1. **for** value **in** sequence:
2. { code block }

In this case, the variable value is used to hold the value of every item present in the sequence before the iteration begins until this particular iteration is completed.

Loop iterates until the final item of the sequence are reached.

Example

```
primes =  [2, 3, 5, 7]
for x in primes:
    print(x)
```

Print each fruit in a fruit list:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```

The for loop does not require an indexing variable to set beforehand.

Looping Through a String

Even strings are iterable objects, they contain a sequence of characters:

Example

Loop through the letters in the word "banana":

```
for x in "banana":
  print(x)
```

The break Statement

With the break statement we can stop the loop before it has looped through all the items:

Example

Exit the loop when x is "banana":

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```

Example

Exit the loop when x is "banana", but this time the break comes before the print:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    break
  print(x)
```

The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next:

Example

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

The range() Function

To loop through a set of code a specified number of times, we can use the range() function,

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

it generates a list of numbers, which is generally used to iterate over with for loop. range() function uses three types of parameters, which are:

- start: Starting number of the sequence.
- stop: Generate numbers up to, but not including last number.

- step: Difference between each number in the sequence.

Python use range( ) function in three ways:

a. range(stop)

b. range(start, stop)

c. range(start, stop, step)

Note:

- All parameters must be integers.
- All parameters can be positive or negative.

Example

Using the range() function:

```
for x in range(6):
  print(x)
```

Note that range(6) is not the values of 0 to 6, but the values 0 to 5.

The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6):

Example

Using the start parameter:

```
for x in range(2, 6):
  print(x)
```

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, **3**):

Example

Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):
  print(x)
```

Else in For Loop

The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

Example

Print all numbers from 0 to 5, and print a message when the loop has ended:

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

Note: The else block will NOT be executed if the loop is stopped by a break statement.

Example

Break the loop when x is 3, and see what happens with the else block:

```
for x in range(6):
  if x == 3: break
  print(x)
else:
  print("Finally finished!")
```

Nested Loops

A nested loop is a loop inside a loop.

The "inner loop" will be executed one time for each iteration of the "outer loop":

Example

Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
  for y in fruits:
    print(x, y)
```

The pass Statement

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

Example

```
for x in [0, 1, 2]:
  pass
```

Programs related to Conditional, looping and jumping statements

1. Write a program to check a number whether it is even or odd.

```
num=int(input("Enter the number: "))
if num%2==0:
    print(num, " is even
number") else:
    print(num, " is odd number")
```

2. Write a program in python to check a number whether it is prime or not.

```
num=int(input("Enter the number: "))
for i in range(2,num):
    if num%i==0:
        print(num, "is not prime number")
        break;
else:
    print(num,"is prime number")
```

3. Write a program to check a year whether it is leap year or not.

```
year=int(input("Enter the year: "))
if year%100==0 and year%400==0:
    print("It is a leap year")
elif year%4==0:
    print("It is a leap year")
else:
    print("It is not leap year")
```

4. Write a program in python to convert °C to °F and vice versa.

```
a=int(input("Press 1 for C to F \n Press 2 for F to C \n"))
if a==1:
        c=float(input("Enter the temperature in degree Celsius:  "))
        f= (9/5)*c+32
        print(c, "Celsius = ",f," Fahrenheit")
elif a==2:
        f=float(input("Enter the temperature in Fahrenheit:  "))
        c= (f-32)*5/9
        print(f, "Fahrenheit = ",c," Celsius")
else:
        print("You entered wrong choice")
```

5. Write a program to check a number whether it is palindrome or not.

```
num=int(input("Enter a number : "))
n=num
res=0
while num>0:
    rem=num%10
    res=rem+res*10
    num=num//10
if res==n:
    print("Number is Palindrome")
else:
    print("Number is not Palindrome")
```

6. A number is Armstrong number or not.

```
num=input("Enter a number : ")
length=len(num)
n =int(num)
num=n
sum=0
while n>0:
    rem=n%10
    sum=sum+rem**length
    n=n//10
if num==sum:
    print(num, "is armstrong number")
else:
    print(num, "is not armstrong number")
```

7. To check whether the number is perfect number or not

```
num=int(input("Enter a number : "))
sum=0
for i in range(1,num):
     if (num%i==0):
          sum=sum+i
     if num==sum:
          print(num, "is perfect number")
     else:
       print(num, "is not perfect number")
```

8. Write a program to print Fibonacci series.

```
n=int(input("How many numbers : "))
first=0
second=1
i=3
print(first, second, end=" ")
while i<=n:
    third=first+second
    print(third, end=" ")
    first=second
    second=third
    i=i+1
```

9. To print a pattern using nested loops

```
for i in range(1,5):
    for j in range(1,i+1):
        print(j," ", end=" ")
        print('\n')
```

```
1
1 2
1 2 3
1 2 3 4
```

Strings: Creating and Storing Strings, Basic String Operations, String Slicing and Joining, String Methods, Formatting Strings

INTRODUCTION

When a sequence of characters is grouped together, a meaningful string is created. Thus, a string is a sequence of characters treated as a single unit

THE str CLASS

Strings are objects of the str class. We can create a string using the constructor of str class as:
S1=str() #Creates an Empty string Object
S2=str("Hello") #Creates a String Object for Hello

An alternative way to create a string object is by assigning a string value to a variable.
Example
S1 = "" # Creates a Empty String
S2= "Hello" # Equivalent to S2=str("Hello")

BASIC INBUILT PYTHON FUNCTIONS FOR STRING

Python has several basic inbuilt functions that can be used with strings. A programmer can make use of **min()** and **max()** functions to return the largest and smallest character in a string. We can also use **len()** function to return the number of characters in a string.

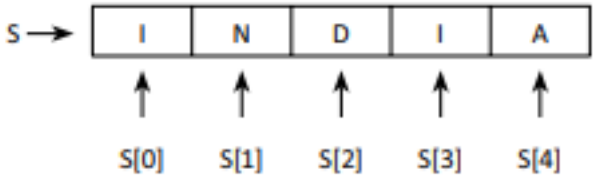
The following example illustrates the use of the basic function on strings.

```
>>> a = "PYTHON"
>>> len(a) #Return length i.e. number of characters in string a
6
>>> min(a) #Return smallest character present in a string
'H'
>>> max(a) #Return largest character present in a string
'Y'
```

THE index[] OPERATOR

As a string is a sequence of characters, the characters in a string can be accessed one at a time through the index operator. The characters in a string are zero based, i.e. the fi rst

character of the string is stored at the 0th position and the last character of the string is stored at a position one less than that of the length of the string.



Accessing characters in a string using the index operator

Example

>>> S1=”Python”

>>>S1[0] #Access the first element of the string.

‘P’

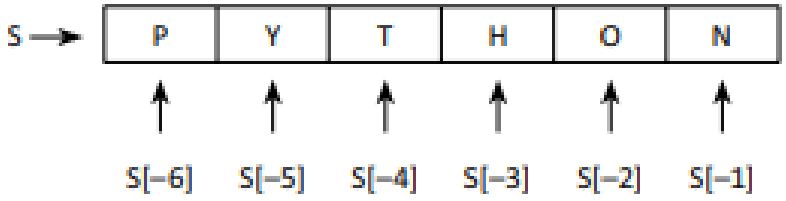
>>>S1[5] #Access the last element of the String.

‘n’

Note: Consider a string of length ‘n’, i.e. the valid indices for such string are from 0 to n-1. If you try to access the index greater than n-1, Python will raise a ‘string index out of range’ error.

Accessing Characters via Negative Index

The negative index accesses characters from the end of a string by counting in backward direction. The index of the last character of any non-empty string is always -1.



2 Accessing characters in a string using negative index

Example

>>> S=”PYTHON”

>>> S[-1]#Access the last character of a String ‘S’

‘N’

>>> S[-2]

‘O’

```
>>> S[-3]
'H'
>>> S[-4]
'T'
>>> S[-5]
'Y'
>>> S[-6]#Access the First character of a String 'S'
'P'
```

TRAVERSING STRING WITH for AND while LOOP

A programmer can use the for loop to traverse all characters in a string. The following code displays all the characters of a string.

PROGRAM

```
S="India"
for ch in S:
    print(ch,end="")
```

Output

India

PROGRAM

```
S="India"
index=0
while index <len(S):
        print(S[index],end="")
        index=index+1
```

Output

India

IMMUTABLE STRINGS

Character sequences fall into two categories, i.e. mutable and immutable. Mutable means changeable and immutable means unchangeable. Hence, strings are immutable sequences of characters.

Example

```
Str1="I Love Python"
```

Str1[0]=”U”
print(Str1)
ERROR: TypeError: ‘str’ object does not support item assignment.

THE STRING OPERATORS

The String Slicing Operator [start: end] The slicing operator returns a subset of a string called slice by specifying two indices, viz. start and end.
The syntax used to return a subset of a string is:
Name_of_Variable_of_a_String[Start_Index: End_Index]

Example

>>> S=”IIT-BOMBAY”
>>> S[4:10] #Returns a Subset of a String
‘BOMBAY’

Name_of_Variable_of_a_String[Start_Index:End_Index:Step_Size]

Example
>>>S=”IIT-BOMBAY”
>>> S[0:len(S):2]
>>>’ITBMA’

Some More Complex Examples of String Slicing
>>> S=”IIT-MADRAS”
>>> S[::] #Prints the entire String
‘IIT-MADRAS’
>>> S[::-1] #**Display the String in Reverse Order**
‘SARDAM-TII’
>>> S[-1:0:-1] #Access the characters of a string from index -1
’SARDAM-TI’
>>> S[:-1] #start with the character stored at index 0 & exclude the last character stored at index -1.
‘IIT-MADRA’

The String +, * and in Operators

1. The + Operator:

The concatenation operator '+' is used to join two strings.

Example:

```
>>> S1="IIT " #The String "IIT" assigned to S1
>>> S2="Delhi" #The String "Delhi" assigned to S2
>>> S1+S2
'IIT Delhi'
```

2. The * Operator:

The multiplication (*) operator is used to concatenate the same string multiple times. It is also called repetition operator.

Example:

```
>>> S1="Hello"
>>> S2=3*S1 #Print the String "Hello" three times
>>> S2 'HelloHelloHello'
```

Note: S2=3*S1 and S2=S1*3 gives same output 3.

The in and not in Operator:

Both Operators in and not in are used to check whether a string is present in another string.

Example:

```
>>> S1="Information Technology"

#Check if the string "Technology" is present in S1
>>> "Technology" in S1
True

#Check if the string "Engineering" is present in S1
>>> "Engineering" in S1
False

# Check if the string "Hello" is not present in S1
>> "Hello" not in S1
True
```

String Comparison

Operators such as ==,,<=,>=and != are used to compare the strings. Python compares strings by comparing their corresponding characters.

Example

>>> S1=”abcd”

>>> S2=”ABCD”

>>> S1>S2

True

The String .format() Method()

Programmers can include %s inside a string and follow it with a list of values for each %.

Example

>>> “My Name is %s and I am from %s”%(“JHON”,”USA”)

‘My Name is JHON and I am from USA’

Python 3 has added a new string method called format() method. Instead of % we can use {0}, {1} and so on. The syntax for format() method is:

template.foramt(P0,P1,…….,k0=V0,K1=V1…}

whereas the arguments to the .format() method are of two types. It consists of zero or more positional arguments Pi followed by zero or more keyword arguments of the form, Ki =Vi.

Example >>> ‘{} plus {} equals {}’**.**format (4, 5,’Nine’)

‘4 plus 5 equals Nine’

Example >>> ‘{1} plus {0} equals {2}’**.**format (4, 5,’Nine’)

‘5 plus 4 equals Nine’

Example >>> “I am {0} years old.I Love to work on {PC} Laptop”.format(25,PC=”APPLE”)

‘I am 25 years old.I Love to work on APPLE Laptop’

The split() Method

The split() method returns a list of all the words in a string. It is used to break up a string into smaller strings.

```
>>>Str1=”C C++ JAVA Python” #Assigns names of Programming languages to Str1
>>>Str1.split()
[‘C,C++,JAVA,Python’]
```

Testing String

A string may contain digits, alphabets or a combination of both of these. Thus, various methods are available to test if the entered string is a digit or alphabet or is alphanumeric.

bool isalnum()

Returns True if characters in the string are alphanumeric and there is at least one character.

Example:

```
>>>S=”Python Programming”
>>>S.isalnum()
False

>>> S=”Python”
>>>S.isalnum()
True
```

bool isalpha()

Returns True if the characters in the string are alphabetic and there is at least one character.

Example:

```
>>> S=”Programming”
>>>S.isalpha()
True

>>> S=”1Programming”
>>>S.isalpha()
False
```

bool isdigit()

Returns True if the characters in the string contain only digits.

Example:

```
>>> Str1=”1234”
>>> Str1.isdigit()
True
>>> Str2=”123Go”
>>> Str2.isdigit()
False
```

bool islower()

Returns True if all the characters in the string are in lowercase.

Example:

```
>>> S=”hello”
>>>S.islower()
True
```

bool isupper()

Returns True if all the characters in the string are in uppercase.

Example:

```
>>> S=”HELLO”
>>>S.isupper ()
True
```

bool isspace()

Returns true if the string contains only white space characters.

Example:

```
>>> S=” “ >>>S.isspace()
True
>>> Str1=”Hello Welcome to Programming World”
>>> Str1.isspace ()
False
```

Searching Substring in a String

Methods of str Class for Searching the Substring in a Given String	*Meaning*

bool endswith(str Str1)
Example:
`>>> S="Python Programming"`
`>>>S.endswith("Programming")`
`True`
	Returns true if the string ends with the substring Str1.

bool startswith(str Str1)
Example:
`>>> S="Python Programming"`
`>>>S.startswith("Python")`
`True`
	Returns true if the string starts with the substring Str1.

int find(str Str1)
Example:
`>>> Str1="Python Programming"`
`>>> Str1.find("Prog")`
`7#Returns the index from where the string "Prog" begins`
`>>> Str1.find("Java")`
`-1#Returns -1 if the string "Java" is not found in the string str1`
	Returns the lowest index where the string Str1 starts in this string or returns -1 if the string Str1 is not found in this string.

int rfind(str Str1)
Example:
`>>> Str1="Python Programming"`
`>>> Str1.rfind("o")`
`9#Returns the index of last occurrence of string "o" in Str1`
	Returns the highest index where the string Str1 starts in this string or returns -1 if the string Str1 is not found in this string.

int count(str S1)
Example:
`>>> Str1="Good Morning"`
`>>> Str1.count("o")`
`3`
	Returns the number of occurrences of this substring.

Methods to Convert a String into Another String

str replace (str old, str new [,count])
Example:
`>>> S1="I have brought two chocolates, two cookies and two cakes"`
`#Replace the old string i.e "two" by new string i.e. "three".`
`>>> S2=S1.replace("two","three")`
`#Replace all occurrences of old string "two" by "three"`
`>>> S2`
`'I have brought three chocolates, three cookies and three cakes'`
	Returns a new string that replaces all the occurrences of the old string with a new string. The third parameter, i.e. the count is optional. It tells the number of old occurrences of the string to be replaced with new occurrences of the string.

Q. Replace two chocolates and two cookies by three chocolates and three cookies.

```
>>> S1="I have brought two chocolates, two cookies
and two cakes"
>>> S1.replace("two","three",2)
#Replace only first 2 occurrences of old string
"two" by "three"
'I have brought three chocolates, three cookies and
two cakes'
```

Methods of Str Class to Convert a String from One Form to Another	*Meaning*

str capitalize()
Example:
`>>> Str1="hello"`
`>>> Str1.capitalize ()`
`'Hello' #Convert first alphabet of String Str1 to uppercase`
	Returns a copy of the string with only the first character capitalised.

str lower()
Example:
`>>> Str1="INDIA"`
`>>> Str1.lower ()`
`'india'`
	Returns a copy of the string with all the letters converted into lower case.

str upper()
Example:
`>>> Str1="iitbombay"`
`>>> Str1.upper()`
`'IITBOMBAY'`
	Returns a copy of the string with all the letters converted into upper case.

str title()
Example:
`>>> Str1="welcome to the world of programming"`
`>>> Str1.title()`
`'Welcome To The World Of Programming'`
	Returns a copy of the string with the first letter capitalised in each word of the string.

str swapcase()
Example:
`>>> Str1="IncreDible India"`
`>>> Str1.swapcase ()`
`'iNCREdIBLE iNDIA'`
	Returns a copy of the string which converts upper case characters into lower case characters and lower case characters into upper case characters.

Stripping Unwanted Characters from a String

A common problem when parsing text is leftover characters at the beginning or end of a string. Python provides various methods to remove white space characters from the beginning, end or both the ends of a string.

Note: Characters such as ‘’, \f,\r, and \n are called white space characters.

str lstrip()

Returns a string with the leading white space characters removed.

str rstrip()

Returns a string with the trailing white space characters removed.

str strip()

Returns a string with the leading and trailing white space characters removed.

Example:

>>> Str1=" Hey,How are you!!!\t\t\t "

>>> Str1 #Print string str1 before stripping

' Hey,How are you!!!\t\t\t '

>>> Str1.strip() #Print after Stripping

'Hey,How are you!!!'

Formatting String

str center(int width)

Returns a copy of the string centered in a fi eld of the given width.

str ljust(int width)

Returns a string left justifi ed in a fi eld of the given width.

str rjust(int width)

Returns a string right justifi ed in a fi eld of the given width.

Example

>>> S1="APPLE MACOS" #Place the string S1 in the center of a string with 11 characters

>>> S1.center(15)

'APPLE MACOS'